

VendorPulse — Client Meeting Prep Pack

Decoding the Solution Architecture & Infrastructure docs · the Foundry story · deployment models · VM sizing · full glossary · anticipated Q&A

Prepared for the Zensar team · for the prospective-client review (Product Owner · Solution Architect · AI Architect) · Confidential

0. Read this first — the 7 things to remember

If you remember nothing else walking into the room, remember these. Everything later just backs them up.

What it is: VendorPulse automates the **Quarterly Business Review (QBR)** process for vendor governance — a fixed **12-step workflow** that the software drives, with AI used only to **draft text**.

Deterministic-first, AI-second: every real decision (ranking slots, validating scores, moving the workflow forward) is **plain code**. The AI never decides or acts — it only writes drafts a human approves. This is the single most important sentence for an AI-governance client.

Human-approval gate (HITL): nothing leaves the system — no email, no invite — until a person clicks Approve. The AI cannot send anything by itself.

Where the AI runs: **Azure AI Foundry, inside Shell's own tenant** (the GPT-4o model), reached over a **private** connection. No data goes to the public internet or trains any model.

How it's hosted: **100% cloud, in Shell's own Azure subscription. Not on-premises. Not serverless.** The current design runs the app on a single **Azure virtual machine (D4as v6 / D8as v6)** inside a private network.

What changed recently: we started by calling the model directly from FastAPI; on your steer we moved to the **Azure AI Foundry + Microsoft Agent Framework (MAF)** approach. A working **proof of concept is done**.

No GPU anywhere: all the heavy AI compute happens remotely in Foundry, so neither the server nor the developer laptops need a graphics card.

***ONE-LINE ANSWER if asked "is this serverless / on-prem / cloud?"** — "It runs entirely in Shell's own Azure cloud — single-tenant, inside your region and your network. It is **not** on-premises (no hardware in a Shell building) and the current design is **not** serverless: it's a right-sized **Azure virtual machine** we fully control, which keeps the security review simple. We can move it to a managed/serverless Azure service later without changing the code."*

1. The project in plain English

VendorPulse is a **vendor-governance workflow app**. Large organisations run **QBRs** — Quarterly Business Reviews — with their key vendors: a structured meeting where both sides review how the vendor performed (scorecards), agree actions, and prepare for the next quarter. Coordinating one is a lot of manual chasing: find a slot everyone can make, send invites, collect scorecards, compile them, brief leadership, prep for the meeting, capture minutes. VendorPulse turns that into a guided, auditable assembly line.

The work moves through a **12-state workflow** (created → attendees refreshed → availability collected → meeting scheduled → scorecards requested → collected → compiled → internal alignment → vendor prep → meeting → post-meeting → archived). The workflow can only move **forward, one step at a time** — it can never skip or go backwards. That rigidity is a feature: it makes the process predictable and auditable, which is exactly what a governance tool needs.

There are **6 functional modules (A–F)** — Scheduling, Scorecard, Alignment, Vendor Prep, Meeting, Analytics. Each has an **AI agent** that drafts the human-readable text for that step (an invite, a brief, meeting minutes, a leadership summary). The agent **never** does the maths or makes the decision — that's deterministic code. It only writes the words, and a human approves them.

*The mental model to carry into the room — Think of VendorPulse as a **factory conveyor belt with a smart assistant standing next to it**. The belt (the workflow) is rigid and deterministic. The assistant (the AI) can draft a nicely-worded email or summary when asked, but it can't move the belt, change a number, or post anything — a human supervisor signs off every output. Governance clients buy the conveyor belt; the AI is a convenience on top, fully fenced in.*

2. The two shared documents, decoded

2.1 Solution Architecture — the 5 numbered layers

The architecture diagram is read **left to right in 5 zones**. Here is what each one means in plain language:

#	Zone	What it actually means
1	Client	The user's web browser . Three kinds of user — VMO Coordinator, Sponsor, Viewer — all log in with their normal Shell account (Entra SSO). Nothing is installed on their machine.
2	Edge	Azure Front Door + WAF — the secure front gate. Terminates HTTPS, applies a Web Application Firewall (blocks common web attacks), and only lets traffic through to our server ("origin-lock").
3	Shell Azure Subscription	Shell's own slice of Azure, inside the approved region . Here lives a private network (VNet) containing the Azure VM that runs the whole app: the React screen, login/permissions, the 12-state WorkflowEngine, the approval gate , the GraphService, and the MAF agent layer that talks to Foundry.
4	Data Tier	The app's private back room, reachable only over Private Link (no public access): PostgreSQL (database), Key Vault (passwords & certificates), Blob Storage (minutes/transcripts), App Insights + Log Analytics (audit trail), and Azure AI Foundry (GPT-4o) — all inside Shell's tenant.
5	External	The only things reached outside, over outbound HTTPS via Shell's egress proxy: Microsoft Graph (send Outlook mail, manage calendars, create Teams meetings) and Entra ID (verify who the user is).

***Watch-out (be honest if asked)** — The diagram shows an **Azure App Service** icon at the top of zone 3 but labels the compute box **Azure VM**, and the Infrastructure doc specifies a VM (D4as v6 / D8as v6). If a sharp architect spots this, the honest answer is: "the compute target is an Azure VM; the App Service icon is a leftover from an earlier draft. Both are valid Azure hosting options and the application code is identical either way." See §4 for why VM was chosen.*

2.2 Infrastructure & Software Requirements — what it lists

This is the shopping list: cloud services, software versions, licences, and the access Shell must provision. The headline entry — and the one leadership keeps asking about — is the very first row: the backend runs on an **Azure VM, size D4as v6 or D8as v6** (covered in §4 and §5). Everything else (Static Web Apps, PostgreSQL, Key Vault, Front Door, Foundry, Graph) matches the architecture diagram. The software section pins exact versions (Python 3.11, FastAPI 0.115.6, React 19, the MAF SDK ≈1.8, etc.) so the build is reproducible and passes Shell's security scanning.

3. The Foundry story — where we came from, where we are

You will be asked why the design mentions Foundry, MAF, and an abstraction layer. Here is the honest, simple narrative.

Stage	What we did	Why it matters to the client
Start (PoC v1)	A FastAPI backend that called the model directly — raw calls to the Foundry / Azure OpenAI Responses API , with our own hand-written tool-calling loop.	Proved the idea, but every control (approval, safety, logging) was hand-rolled — more code for Shell's security team to review.
Your steer (last meeting)	Use Azure AI Foundry properly — build on the Microsoft Agent Framework (MAF) SDK over the Foundry Responses API, not raw calls.	Foundry/MAF provide platform-built human-in-the-loop approval, content safety, and tracing — fewer hand-rolled controls, easier to clear Shell's review.
Now (PoC v2 — done)	A working PoC on MAF + Foundry Responses API across 2 agents, approval gate proven (GitHub issue #13). You may see a branch named <code>po c/scheduling-foundry-response s</code> .	De-risks the build: the agentic pattern, tool-calling, and approval gate are proven on real Foundry before any production spend.
Decision	Production builds on the MAF SDK , with the direct-Responses path kept as fallback. An LLMProvider abstraction keeps Anthropic/Claude selectable.	We're not locked to one vendor; the model choice is a config switch, not a rewrite.

The three terms people confuse — say them like this

- Azure OpenAI** = just the **model behind an API** (send text, get text). The plumbing.
- Azure AI Foundry** = the **whole platform** around those models — model catalog, agents, content safety, tracing, deployments. Our GPT-4o deployment lives here, inside Shell's tenant. We reach it through its **Responses API**.
- Microsoft Agent Framework (MAF)** = the **open-source SDK** we write our agents with (`agent_framework.Agent + @tool` functions). MAF is our code; it calls Foundry. It gives us the agent loop, tool-calling, and a built-in approval mode for free.

*If asked "so are you using Azure OpenAI or Azure AI Foundry?" — "Both names describe the same Microsoft stack at different layers. Our agent code is written with the **Microsoft Agent Framework**; it calls the **Azure AI Foundry Responses API**; and the model serving behind that is the **Azure OpenAI GPT-4o** family — all hosted inside Shell's own Foundry tenant."*

4. Deployment models, explained for leadership

This is the section to clear up the on-prem / cloud / serverless confusion. There is a **spectrum** of how much you manage yourself versus how much the cloud manages for you. From most-you to most-cloud:

Model	Plain-English analogy	Who manages what	Is VendorPulse this?
On-premises	You own the house — buy the land, build it, fix the boiler.	You own physical servers in your own building: hardware, power, cooling, patching.	No. Nothing runs in a Shell building. Zero physical hardware.
IaaS — cloud VM (<i>current design</i>)	You rent an empty apartment — building managed, you furnish & clean inside.	Azure owns the hardware; you manage the VM's OS, patching, and app.	Yes — this is it today. An Azure VM (D4as v6/D8as v6).
PaaS — managed platform	You rent a serviced apartment — furniture & cleaning included.	Azure manages OS + runtime; you deploy code/container (App Service, Container Apps).	Not currently , but a valid future move — code unchanged. (Earlier draft used this.)
Serverless / FaaS	You stay in a hotel — pay per night, staff handle everything.	Azure manages everything incl. scaling; pay per execution, scale to zero (Functions).	No. Current design is an always-on VM, not pay-per-call.

So, in one breath

VendorPulse is cloud-hosted IaaS, single-tenant, inside Shell's own Azure subscription and region. It is not on-premises and it is not serverless. It's a virtual machine we control, in a private network, talking to managed Azure services (database, secrets, AI) over private links.

Why a plain VM and not serverless / a managed platform?

- Simplest security review (the big one for Shell):** a single VM in a VNet is the easiest shape to reason about, lock down, and get past IRM / IT-Security. Fewer managed services = fewer things to assess.
- Full control & no preview dependencies:** we own the OS and runtime; nothing depends on a preview-tier feature.
- Predictable cost & behaviour:** an always-on VM has a flat monthly cost and no cold-start latency — fine for an internal governance tool with steady, modest traffic.
- The code is portable:** the app is a standard container/FastAPI app, so moving later to Container Apps (PaaS) or a Foundry Hosted Agent (managed) is a **hosting swap, not a rewrite**.

If they push: "serverless would be cheaper / auto-scale — why not?" — "For a steady internal workload with a human approval step in the loop, elastic scaling and scale-to-zero buy us little, while a single VM is far simpler for your security team to certify. The app is containerised, so if usage grows we move to Azure Container Apps or a Foundry Hosted Agent — same code — and get autoscale then. We chose the option that ships fastest through Shell governance."

5. VM sizing — exactly what to say

The Infrastructure doc specifies an **Azure D4as v6** or **D8as v6** virtual machine. Here is what those names mean and how to defend the choice.

5.1 Decoding the VM name

Part of the name	Meaning
D	D-series = general-purpose VM (balanced CPU-to-memory). Right for a normal web/API workload.
4 / 8	Number of vCPUs (virtual CPU cores): 4 or 8 .

Part of the name	Meaning
a	Runs on AMD (EPYC) processors — strong price/performance.
s	Premium SSD capable — fast, reliable disk.
v6	Generation 6 — the current, efficient hardware generation.

5.2 The actual specs

Size	vCPUs	RAM	Good for	Indicative cost*
D4as v6	4 cores	16 GB	Dev / Staging, and Production at expected QBR volumes (steady, modest concurrent users).	~\$140–200 / month
D8as v6	8 cores	32 GB	Production headroom for many concurrent coordinators, or comfortable margin.	~\$280–400 / month

Cost is pay-as-you-go indicative for West Europe and drops materially with a 1- or 3-year reserved-instance commitment — present as "order of magnitude," not a quote.

5.3 Why this size is right — the reasoning to give

- The heavy lifting isn't here.** AI inference runs remotely in **Foundry**, and the database is a separate managed service. The VM only runs a lightweight **FastAPI** app and MAF orchestration — not CPU- or memory-hungry.
- No GPU needed** — say this proactively, because people assume "AI = expensive GPU server." There is **no GPU** on the VM or on developer laptops; all model compute is Foundry's.
- 4 vCPU / 16 GB comfortably handles** a FastAPI app serving an internal governance team. The workload is bursty-but-small (drafting an invite, approving a brief) — not high-throughput.
- Vertical headroom is trivial:** if it needs more, resizing (e.g. D4→D8) is a few-minute reboot in the portal — no re-architecting.
- Production gets resilience** via Azure zone-redundancy/availability, and the database is **High-Availability** — so the small VM size doesn't compromise uptime.

If asked "what spec / how big a server?" — the crisp answer — "A general-purpose Azure VM — 4 vCPUs and 16 GB RAM (D4as v6) for most environments, scaling to 8 vCPU / 32 GB (D8as v6) in production if we want headroom. No GPU is needed because all AI inference happens in Foundry, and the database is a separate managed service. The VM just runs the FastAPI app — it's a light workload, and we can resize in minutes if volumes grow."

6. The full glossary — every term, in plain language

Grouped so you can find a term fast. For each: what it is, and (where useful) why it's in our design.

6.1 Deployment & hosting

Term	Plain meaning
On-premises / on-prem	Physical servers you own and run in your own building/datacenter. Not used here.

Term	Plain meaning
Cloud / public cloud	Computing rented from a provider (here, Microsoft Azure) — no hardware to own.
IaaS (Infrastructure as a Service)	You rent a virtual machine ; the cloud runs the hardware, you manage the OS + app. VendorPulse's current model .
PaaS (Platform as a Service)	The cloud manages OS + runtime; you deploy code/containers (App Service, Container Apps).
SaaS (Software as a Service)	Finished software you log into (e.g. Microsoft 365). VendorPulse is delivered as an internal app, not sold as SaaS.
Serverless / FaaS	Pay-per-execution compute that scales to zero (Azure Functions). Cloud handles all scaling. Not the current design .
Single-tenant	The whole deployment is dedicated to Shell — Shell's data, tenant, region, no sharing.
Multi-tenant	One shared deployment serving many customers (opposite). Not how this is built.
Virtual Machine (VM)	A computer that exists as software inside Azure's datacenter — full OS control, no owned hardware.
vCPU	A virtual CPU core — the unit of processing power assigned to a VM.
Autoscale / replicas	Automatically adding/removing copies of the app as load changes. A managed/serverless feature; a single VM scales by resizing.
High Availability (HA)	Running redundantly so a single failure doesn't cause downtime. Our PostgreSQL is HA in production.
Zone-redundant	Spread across separate Azure datacenters in a region so one outage doesn't take you down.
Region / data residency	The geographic datacenter location (e.g. West Europe). Keeping data in an approved region is a Shell requirement.
IaC (Infrastructure as Code)	Defining all cloud setup in text files (Bicep / Terraform) so environments are reproducible and reviewable.

6.2 Azure services in the docs

Term	Plain meaning
Azure subscription / resource group	Shell's billing account in Azure / a labelled folder grouping related resources.
Azure VM	The virtual machine running the FastAPI + MAF backend (D4as v6 / D8as v6).
Azure Static Web Apps	A cheap, fast Azure service for hosting the built React front-end via global CDN.
Azure Container Apps / App Service	Managed (PaaS) ways to run the backend without managing a VM — alternatives to the VM, same code.
Azure Container Registry (ACR)	A private store for the app's Docker container images .

Term	Plain meaning
Azure PostgreSQL Flexible Server	The managed database (open-source PostgreSQL). GP = General Purpose tier, Burstable = cheap dev tier.
Azure Key Vault	A secure safe for secrets — passwords, keys, certificates — never in code or config files.
Azure Front Door + WAF	The global secure entry point. WAF (Web Application Firewall) blocks common web attacks using OWASP rules.
OWASP	An industry-standard list of top web-app security risks; the WAF enforces rules against them.
App Insights / Azure Monitor / Log Analytics	Azure's observability stack — logs, metrics, traces; also our immutable audit mirror.
Azure Blob Storage	Cheap object storage for files (future: meeting minutes / transcripts).
VNet (Virtual Network)	A private network in Azure; our VM lives inside it, isolated from the public internet.
Private Endpoint / Private Link	A private, internal-only connection to an Azure service so traffic never crosses the public internet .
Egress proxy	Shell's controlled outbound gateway — the only way the app reaches external services (Graph, Entra).
Origin-lock / Service Tag	Config ensuring the backend only accepts traffic from Front Door, not directly from the internet.
TLS / TLS 1.2+	The encryption protocol behind HTTPS that protects data in transit.

6.3 Identity & access

Term	Plain meaning
Microsoft Entra ID	Microsoft's identity service — formerly Azure Active Directory (Azure AD) . How users log in and how the app proves its identity.
SSO (Single Sign-On)	Users log in with their existing Shell account — no separate VendorPulse password.
OIDC (OpenID Connect)	The standard protocol behind that SSO login.
RBAC (Role-Based Access Control)	Permissions by role — Coordinator vs Viewer — so people only see/do what their role allows.
JWT (JSON Web Token)	The signed digital "badge" a user carries after login to prove identity on each request.
Managed Identity	An automatic, password-less identity Azure gives the VM to access Key Vault, the DB, and Foundry with no stored secret .
MSAL	Microsoft's auth library used for the app-to-Graph login.
App registration	The app's identity record in Entra, defining what it's allowed to do.
Client-credentials flow	App-to-service login (no human) — lets VendorPulse send mail or read calendars unattended.

Term	Plain meaning
Admin consent	A Shell global admin's one-time approval of the permissions the app requests (2–4 week lead time).
Application Access Policy	An Exchange control limiting the app's mail permission to one specific mailbox , not everyone's.
App-only / certificate auth	Production app authenticates as itself using a certificate (stronger than a secret), replacing the PoC's pasted 1-hour token.

6.4 Microsoft Graph & Microsoft 365

Term	Plain meaning
Microsoft Graph	The single API to reach Microsoft 365 — Outlook mail, calendars, Teams meetings, user lookups . Used to send invites and schedule meetings.
Mail.Send / Mail.Read	Graph permissions: send email / read replies.
Calendars.ReadWrite	Graph permission: read free/busy and create calendar events.
OnlineMeetings.ReadWrite.All	Graph permission: create Teams meetings.
User.Read.All	Graph permission: look up users in the directory (for attendee lists).
Microsoft 365 E3 / E5	Microsoft's productivity+security licence tiers; needed for the service mailbox the app sends from. E5 adds advanced security/compliance.

6.5 AI / agent stack

Term	Plain meaning
LLM (Large Language Model)	The AI that generates text (here, GPT-4o). In VendorPulse it only drafts wording — never decides or acts.
GPT-4o / gpt-4.1	Specific OpenAI model versions, deployed inside Shell's Foundry. GA (Generally Available) = stable, production-grade, not preview.
Azure OpenAI	The Azure service that serves OpenAI models behind an API — the model layer.
Azure AI Foundry	Microsoft's platform around those models: model catalog, agents, content safety, tracing. Our GPT-4o lives here, in Shell's tenant.
Responses API	The Foundry/OpenAI API our agents call for model output and tool-calls. Our single entry point.
Microsoft Agent Framework (MAF)	The open-source SDK we build agents with. Provides the agent loop, tool-calling, and built-in approval mode. Our code , calling Foundry.
Agent	A small AI-driven routine for one module (e.g. SchedulingAgent) that, given a prompt, can call a set of tools and return a structured result.

Term	Plain meaning
Tool / tool-calling / @tool	Functions the agent may call ("rank slots", "draft invite"). The model picks; our code runs them. Result is structured data , not free text.
HITL (Human-in-the-loop) / approval_mode	The control where a human must approve before a sensitive action runs. Gated both in the app and natively in MAF (belt-and-suspenders).
Content safety / content filters	Foundry's built-in filters blocking harmful/unsafe model input/output. XPIA = protection against prompt-injection attacks.
OpenTelemetry (OTel) / tracing	An open standard for emitting logs/traces. MAF emits these by default; they flow to App Insights for audit.
Hallucination	When an LLM confidently states something false. Mitigated by keeping all facts/numbers/IDs in deterministic code , not the model.
Deterministic	Code that always gives the same output for the same input (opposite of an LLM's variability). All VendorPulse decisions are deterministic.
LLMProvider abstraction	A config switch that lets us swap the model vendor (Foundry or Anthropic/Claude) without changing application code.
AgentResponse / adapter	The single fixed response shape every agent returns, so the front-end never guesses the output format.
Foundry Hosted Agents	A (preview) Foundry option to host our agent code as a managed endpoint — a possible future upgrade from the VM, same code.
Claude / Anthropic	An alternative LLM vendor, selectable via the abstraction. Claude Code is separately the AI coding assistant the dev team uses to build faster.

6.6 Application & developer stack

Term	Plain meaning
FastAPI	The Python web framework the backend is built on — serves the REST API.
Python 3.11	The backend programming language/version.
Uvicorn / ASGI	The server that runs the FastAPI app.
Pydantic	Library that validates data shapes — guarantees inputs/outputs match the defined schema.
React 19 / SPA	The front-end framework. SPA = Single-Page Application: UI runs in the browser, updates without full reloads.
Vite / TypeScript / Zustand / Recharts / Tailwind	Front-end build tool / typed JavaScript / state management / charts / styling.
WorkflowEngine / 12-state machine	The deterministic core enforcing the forward-only 12-step process — single source of truth for what's allowed next.
Module A–F / the 6 agents	Scheduling, Scorecard, Alignment, Vendor Prep, Meeting, Analytics — each a functional area with its own agent.

Term	Plain meaning
Repository pattern / BaseRepository	The single code layer that touches storage. Swapping JSON files → PostgreSQL means changing only this layer.
PostgreSQL / SQLite / JSON files	The production database / lightweight dev database / original file-based storage being replaced.
SQLAlchemy / Alembic	Database toolkit / migration tool for evolving the schema safely.
Magic-link / one-time token	A secure single-use link emailed to a stakeholder to open the scorecard form without a full account.
CI/CD	Continuous Integration / Delivery — the automated pipeline that lints, tests, builds, scans, and deploys (GitHub Actions / Azure DevOps).
Docker / container / image	Packaging the app with everything it needs to run identically anywhere; the image is the packaged artifact.
SAST / SonarQube / Trivy / GitLeaks	Security scanning in CI: static code analysis / image & dependency scan / secret-leak detection.
ruff / eslint / tsc	Automated code-quality/linting checks for Python and TypeScript.

6.7 Governance & compliance (Shell-specific)

Term	Plain meaning
QBR (Quarterly Business Review)	The recurring vendor-performance meeting VendorPulse orchestrates.
VMO (Vendor Management Office)	The team that runs vendor governance — the primary users.
IRM 3.492	Shell's internal control standard for AI systems — the rulebook this design maps to.
NIST AI RMF / ISO 42001	External AI risk-management / AI-management-system standards that IRM 3.492 is built on.
EU AI Act	European regulation on AI use; requires transparency (e.g. labelling AI-generated content).
AI Registry / ServiceNow / IAQ	Shell's mandatory AI-system registration and risk-assessment (IRM Assessment Questionnaire) — done before production .
Shell.AI / TRB	Shell's AI governance body / Technology Review Board — must approve the model and design before production.
IDT	Shell's Information & Digital Technology org — production must run on IDT-managed Azure/Foundry.
DPA / no-training assurance	Data Processing Agreement; the contractual guarantee that Shell's data is not used to train any model.
Data classification	Labelling data by sensitivity — scorecards likely Commercially Sensitive , attendee data PII/GDPR . (No SOX/export-controlled data in scope.)

7. Anticipated questions & crisp answers

Organised by who's likely to ask. Answers are written to be said out loud.

7.1 From the Product Owner (value, scope, users)

Q. What does this actually save us? — It removes the manual chasing in every QBR — scheduling, scorecard collection, compiling, briefing, minutes — into one guided, auditable workflow. Coordinators approve drafts instead of writing from scratch; leadership gets consistent briefs; nothing falls through the cracks because the workflow enforces every step.

Q. Who are the users and how do they log in? — Three roles — VMO Coordinator, Sponsor, Viewer — all logging in with their normal Shell account via single sign-on. No new passwords, permissions are role-based.

Q. How much of this is AI "making decisions"? — **None.** Every decision — slot ranking, score validation, workflow progression — is deterministic code. The AI only drafts text, and a person approves it before anything happens.

Q. What's live today vs still to build? — A working PoC proves the agentic pattern and the approval gate on real Foundry (2 agents). Production work — full Azure infra, identity hardening, database migration, porting all agents, and clearing Shell's compliance gates — is the build phase ahead.

7.2 From the Solution Architect (hosting, integration, data)

Q. Is this serverless, PaaS, or a VM? And why? — Cloud IaaS — a single right-sized **Azure VM** in a private VNet in Shell's subscription. We chose a VM for the simplest security review and full control; the app is containerised so we can move to Container Apps (PaaS) or a Foundry Hosted Agent later without a rewrite. It is **not** on-prem and **not** serverless today.

Q. How does it talk to Microsoft 365 / our data? — Through **Microsoft Graph**, authenticating app-only with a **certificate**, scoped to a single mailbox via an Application Access Policy. Database, Key Vault, and Foundry are reached over **Private Endpoints** — nothing sensitive crosses the public internet.

Q. Where does our data live, and does it leave our tenant? — Everything stays **single-tenant inside Shell's Azure and chosen region**. Foundry runs in Shell's tenant. Data is not used for model training (confirmed in the DPA). Secrets live in Key Vault, accessed via Managed Identity — none in code.

Q. How do you handle secrets, identity, and audit? — Secrets in **Key Vault** via **Managed Identity** (no stored passwords). Users authenticate with **Entra SSO + RBAC**. Every agent run and action is logged with correlation IDs to an immutable **App Insights / Log Analytics** audit trail.

Q. What about the missing automated tests / tech debt? — Honestly stated: the PoC has no automated suite yet. The production CI pipeline adds a **regression stage** that gates every deploy and must cover the approval gate, deterministic path, and response contract before any agent change ships.

7.3 From the AI Architect (the agentic layer, safety, model)

Q. Why MAF over just calling the model directly? — MAF gives platform-built human-in-the-loop approval, content safety, and on-by-default tracing — so we hand-roll fewer controls, a smaller surface for Shell's code-security review. We proved both the direct-call and MAF paths in the PoC; production builds on MAF, with direct-Responses as fallback.

Q. How do you stop hallucinations from doing damage? — The model never touches facts, numbers, IDs, or decisions — those are deterministic code. The LLM only produces draft prose, grounded in supplied context and **always** human-approved before use. So a hallucination is a wording a reviewer rejects, never a wrong action taken.

Q. Which model, and are we locked into Microsoft? — GPT-4o (GA) deployed in Shell's Foundry today. We're **not locked in**: an LLMProvider abstraction keeps Anthropic/Claude selectable as a configuration choice.

Q. How is the agent prevented from sending things on its own? — Two layers. The **app-layer gate**: side-effecting actions (send invite, send mail) are removed from the agent and only fire from deterministic routes after a human approves. Plus MAF's native **approval_mode** on any in-run tool. The agent drafts; it never sends.

Q. Do you need GPUs / how heavy is the compute? — No GPUs anywhere. All inference is remote in Foundry. The VM only runs a light FastAPI app, so 4–8 vCPUs and 16–32 GB RAM is ample.

7.4 Deployment, cost & timeline

Q. What size server and what does it cost to run? — A general-purpose **D4as v6 (4 vCPU / 16 GB)**, up to **D8as v6 (8 vCPU / 32 GB)** in production. Indicative Azure run-cost is roughly **\$450–650/month** for the production stack plus **~\$1,000/month** of Foundry model usage — order-of-magnitude, lower with reserved instances. Non-prod adds **~\$80–120/month**.

Q. What's the biggest thing standing between us and production? — Not the technology — it's **Shell's own compliance gates**: AI Registry + ServiceNow registration, the IRM risk assessment (IAQ), EU AI Act classification, and Shell.AI + TRB approval, all on IDT-managed Azure. These have external lead time, so start **in parallel, now**.

Q. Can it scale if adoption grows? — Yes — vertically by resizing the VM in minutes, or by moving the containerised app to autoscaling Container Apps / a Foundry Hosted Agent with no code change. The database is already HA and can scale its tier independently.

8. Honest caveats — so you're never caught out

Better you raise these than the client. None are dealbreakers; all have a clean answer.

- Diagram says App Service icon, doc says VM.** Acknowledge it's a draft artifact; the target is a VM; both are valid and the code is identical. (§2.1)
- This is still pre-production.** The PoC proves the pattern; the production build (infra, identity, DB migration, all agents, tests) is ahead. Don't imply it's deployed.
- No automated test suite yet** — the CI regression stage is the plan to fix that before any agent change ships.
- Foundry Hosted Agents is still preview** — mention it only as a *future* hosting option; the shippable baseline is the VM.
- Compliance gates are blocking and external** — production cannot go live until Shell's AI Registry / IAQ / TRB / Shell.AI approvals clear. Frame this as a shared workstream to start now, not a Zensar delay.
- Costs are indicative** — always say "order of magnitude, subject to Shell tiering and usage," never quote them as a price.